RESEARCH UNIVERSITY
EXCELLENCE INITIATIVE



Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

AGH University of Krakow

Numerical Methods in Physics

Project 01
Simple Numerical Operations with Vectors and Matrices

Hamzeh Khanpour
hamzeh.khanpour@fis.agh.edu.pl
view.fis.agh.edu.pl/staff/hamzeh.khanpour/

Faculty of Physics and Applied Computer Science
AGH University of Krakow

October 9, 2026

The main goal of this laboratory

More than multiplying matrices

The goal of this laboratory is **not simply to multiply matrices**. It is to understand how mathematical operations become **numerical algorithms**, how their results behave under **repeated computation**, and how to implement them **accurately and efficiently in C++**.

Scientific computing in practice

You will gain your first practical experience with **scientific computing** by combining mathematical analysis, algorithm development, numerical experimentation, data visualization, and computational performance evaluation.

To do list ...

Matrix multiplication is our starting example, not the whole lesson. We will turn a mathematical rule into C++, observe what happens when the calculation is repeated, and use recorded results and plots to explain that behaviour. We will also examine correctness and computational efficiency. Emphasize that writing a program, understanding its output, and evaluating its performance are connected parts of scientific computing.

AGH

Two questions

How do we turn vector and matrix formulas into C++?

How do we know that the results are reliable and the comparison is fair?

Scientific workflow

Question $\longrightarrow$ method $\longrightarrow$ implementation $\longrightarrow$ verification $\longrightarrow$ interpretation.

Course sources

S01: Tomasz Chwiej, Numerical Methods in Physics course page.
S02: Project 01 handout, 3 March 2026.
S03: Lecture 1, Numerical Errors.

Technical sources

Netlib BLAS and Mark Gates' CBLAS reference; GCC 14.2 manuals; C++ working-draft clock/type sections; N. J. Higham on backward error.

C03 | Outcomes and required deliverables

AGH

The numerical work

Build the 2000×2000 matrix.
Apply it to a vector 30 times.
Repeat for each gamma; save and plot both norm histories.

The omparison

Compute $C = A^2$ with your own loops and CBLAS.
Check the answers, then measure times.
Explain the observed behaviour.

Two submission tracks

Activity: source files and required data as ZIP.
Report: a separate scientific PDF. Precision and compiler-flag studies are optional extensions.

www.agh.edu.pl



```
hostname
pwd
g++ --version
make basic
make check-env
```

Know where this command runs

Personal Linux, a classroom PC and Taurus can share files but have different software. Run these commands from the **student starter directory**.

When CBLAS is unavailable

Continue the library-free activities. Record the machine, command and first error; ask the instructor. This does not complete the official BLAS task. Do not install packages on a managed machine.

AGH

Task: what result do we want?

Given A and x, calculate $y = Ax$.

Method: what mathematical operation gives it?

For each row, evaluate $y_i = \sum_{j=0}^{n-1} A_{ij} x_j$.

Algorithm: how will the computer do it?

Choose a layout; allocate separate buffers; reset each sum; traverse the indices; store each result; check it. These decisions are not specified by the symbol Ax alone.

$$u^T v = \sum_{i=0}^{n-1} u_i v_i$$

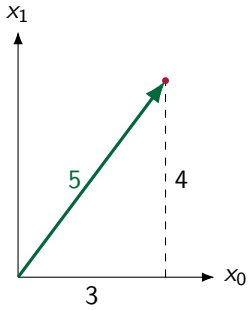
i	0	1	2
u_i	1	2	3
v_i	4	5	6
$u_i v_i$	4	10	18

Accumulate every contribution

$u^T v = 4 + 10 + 18 = 32$. The result is one scalar, not a vector. Both inputs must have the same length.

```
double sum = 0.0;
for (std::size_t i = 0; i < u.size(); ++i)
    sum += u[i] * v[i];
```

$$\|x\|_2 = \sqrt{\sum_i x_i^2}$$



A complete tiny example

$x = (3, 4)^T$ gives $\|x\|_2 = \sqrt{9 + 16} = 5$. The zero vector has norm zero. Reversing a component sign does not change the norm.

Numerical caution

Directly squaring very large values can overflow even when the norm is representable. R05 explains a scaled implementation.

AGH

Prediction P1

Let $u = (2, -1, 0)^T$ and $v = (3, 4, 5)^T$. Before running code, write down $u^T v$, $\|u\|_2$ and $\|v\|_2$. Show every contribution.

A useful comparison

For $w = (1, 2, 3)^T$, $w^T w = 1 + 4 + 9 = 14$, so $\|w\|_2 = \sqrt{14}$. The dot product with itself is the *squared* norm.

Discuss P2

Would permuting the components of both vectors in the same way change the exact dot product? Could the floating-point result change in its last digits?

```cpp
std::vector<double> a(6, 0.0);
// Six contiguous double values; all initially zero.
std::size_t count = a.size();
double* pointer = a.data();
```

a_0	a_1	a_2	a_3	a_4	a_5
0	1	2	3	4	5

Logical shape is additional information

The same six values could represent a 2×3 or 3×2 matrix. Store the row count, column count, layout and leading dimension with the data.

A pointer passed to CBLAS does not carry a matrix shape. Its size and strides must be supplied correctly.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8

Zero-based offset

$$\boxed{\text{offset}(i, j) = i\,\text{lda} + j}$$

For packed 3×3 storage, $\text{lda} = 3$. Thus $A_{2,1} = 8$ is at offset $2 \cdot 3 + 1 = 7$.

Increasing j by one moves to the adjacent element in the same row.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

1	4	7	2	5	8	3	6	9
0	1	2	3	4	5	6	7	8

Zero-based offset

$$\boxed{\text{offset}(i, j) = i + j\,\text{lda}}$$

For packed 3×3 storage, $\text{lda} = 3$. Thus $A_{2,1} = 8$ is at offset $2 + 1 \cdot 3 = 5$.

The logical matrix has not been transposed. Only its representation in memory changed.

1	2	3	pad	pad	offsets 0–4
4	5	6	pad	pad	offsets 5–9
7	8	9	pad	pad	offsets 10–14

Logical width 3; stored row stride 5

$\mathrm{lda} = 5$ counts **elements**, not bytes. The next row begins five elements later. Padding is not part of the dot product.

Check an offset

$A_{2,1}$ is at $2 \cdot 5 + 1 = 11$. Allocate $3 \cdot 5 = 15$ storage slots in this representation, not just 3^2.

AGH

One official matrix

$n^2 = 4{,}000{,}000$ entries.
For eight-byte doubles:
32,000,000 bytes
≈ 30.52 MiB.
Three such matrices use about 91.55 MiB before other buffers.

Different operations

Dot product: $O(n)$.
Matrix–vector: $O(n^2)$.
Matrix–matrix: $O(n^3)$.
At $n = 2000$, the square contains about $2n^3 = 16$ billion leading-order floating-point operations.

Safety before scale

Check positive dimensions and multiplication overflow before allocation. Put large matrices on managed heap storage, not a small automatic stack.

AGH

A1a–b: implement and check

In `include/student_tasks.hpp`, implement `dot` and `norm`. Test the given dot product, $(3, 4)$, and a zero vector. Add a length-mismatch check.

A1c–d: address logical entries

Implement both offset functions. For a logical 3×3 matrix with $\mathrm{lda} = 5$, explain the offsets of $(2, 1)$ and identify unused slots.

```
make activity1
./bin/activity1
```

A compiled template is not a solution: unfinished functions deliberately report `NOT IMPLEMENTED`. Record your small checks before moving on.

C15 | One row gives one output component

$$\underbrace{A}_{m\times n}\ \underbrace{x}_{n\times 1} = \underbrace{y}_{m\times 1}, \qquad y_i = \sum_{j=0}^{n-1} A_{ij}x_j$$



Compatible dimensions

Each row has exactly as many entries as the input vector. The number of rows determines the output length. Here $n = 2000$ and both vectors have length n.

C16 | Accumulate, then store

```
for (std::size_t i = 0; i < n; ++i) {
    double sum = 0.0;
    for (std::size_t j = 0; j < n; ++j) {
        sum += a[i * lda + j] * x[j];
    }
    y[i] = sum;
}
```

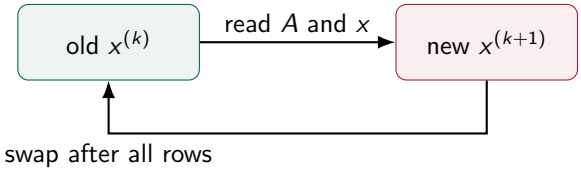
Three different roles

i selects an output row; *j* selects a contribution; `sum` accumulates that row. Reset `sum` once per row. Assign `y[i]` after the sum is complete.

Handout clarification

The displayed pseudocode on handout p.3 omits accumulation. Its defining equation is a sum; implement the sum, not the overwrite.

C17 | Two buffers preserve the old iterate




```
student::matvec(a, n, x, y); // read x, write y
x.swap(y);                   // x now holds the new iterate
// Next product overwrites all entries of y.
```

Not an in-place operation

Writing directly into `x[i]` can make a later row use a mixture of old and new components. That computes a different recurrence.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad x^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

First multiplication

$$x^{(1)} = \begin{pmatrix} 1 \cdot 1 + 2 \cdot 2 \\ 3 \cdot 1 + 4 \cdot 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$$

$$\|x^{(1)}\|_2 = \sqrt{146}$$

Second multiplication

$$x^{(2)} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 11 \\ 3 \cdot 5 + 4 \cdot 11 \end{pmatrix} = \begin{pmatrix} 27 \\ 59 \end{pmatrix}$$

Use *both* components of the old iterate.

A two-step test detects state/buffer errors that a one-step test may miss.

AGH

A2a–b: implement and test

Complete `matvec` in the student header. Use packed row-major storage. Check dimensions and keep input/output separate. Run the two-step fixture with nonzero prefilled output.

```
make activity2
./bin/activity2
```

A2c–d: diagnose and explain

Explain the roles of both loops and the buffer swap. Add one dimension-failure test. Predict the incorrect first result from an in-place update before repairing it.

Completion means correct values *and* a reasoned explanation of the implementation.

The inputs

$$A_{ij}(\gamma) = \frac{1}{|i-j|^4 + \gamma}, \quad i,j = 0,\ldots,1999$$

$$n = \mathrm{lda} = 2000, \qquad \gamma \in \{2,3\}, \qquad x_i^{(0)} = 1$$

What changes between the two runs?

The parameter changes every matrix entry. Rebuild A and reset $x^{(0)}$ to ones for each case. Do not normalize the iterates.

AGH

Distance from the diagonal

$\lvert i-j\rvert$	$\gamma=2$	$\gamma=3$
0	1/2	1/3
1	1/3	1/4
2	1/18	1/19

Real, symmetric, Toeplitz and entrywise positive.

Do not over-interpret

Positive entries do not by themselves imply positive definiteness. Toeplitz means constant diagonals; it does not mean periodic/circulant.

```
const T d = static_cast<T>(i) - static_cast<T>(j);
const T d2 = d * d;
const T denominator = d2 * d2 + gamma;
return T(1) / denominator;
```

Convert *before* subtraction. At distance 1999, the fourth power exceeds a 32-bit integer range.

The recurrence

$$x^{(k+1)} = Ax^{(k)}, \qquad k = 0, \ldots, 29$$

The first saved row describes $x^{(1)}$; the last describes $x^{(30)}$.

Algorithmic route

Initialize x to ones and allocate y.
For each step: compute $y = Ax$, check finiteness, swap buffers, calculate the norm, and save one row.

Two operations to avoid

Do not form the dense matrix A^{30}. Do not divide x by its norm: that would destroy the required norm history.

```
iteration,norm2
1,58.866731383699808
2,77.494999519865544
3,102.02261712302925
```

Two separate files

`norms_gamma2.csv` `norms_gamma3.csv`
Each has the header above and 30 post-step rows. The excerpt is from the measured $\gamma = 2$ run.

Preserve the experiment

Record n, γ, step count, type, compiler and command in a README or metadata file. Write with `max_digits10`; extra printed digits are not a claim of extra accuracy.

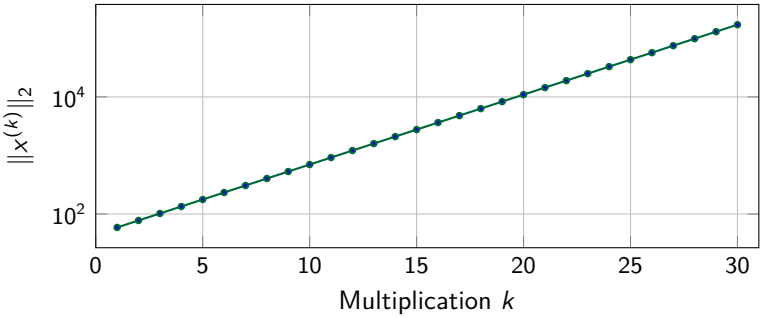
AGH

A3a–d

Complete the matrix builder and norm-history driver in the student header. Test $n = 2$ first. Then run the official $n = 2000$, both γ values, and 30 steps. Preserve two buffers and separate initialization.

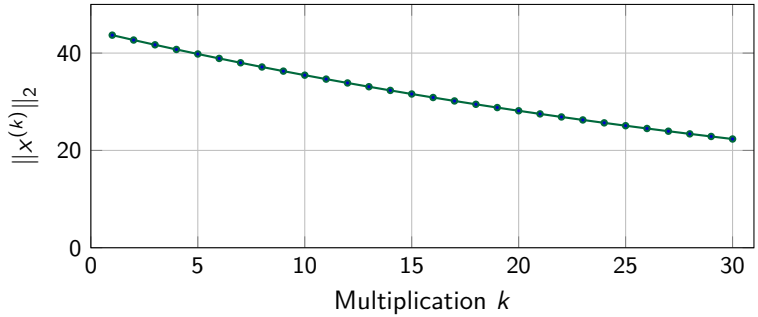
```
make activity3
./bin/activity3 --n 2 --steps 30 --gamma both \
    --output data/check_n2
./bin/activity3 --n 2000 --steps 30 --gamma both \
    --output data/official
python3 plotting/plot_norms.py data/official --output plots
```

Completion: two finite 30-row histories, two labelled plots, and an explanation of the trends. For $n = 2$, both cases decay; do not confuse this check with the official size.



$n = 2000$, $x_i^{(0)} = 1$, 30 C++ binary64 products. Final norm: 171,507.0793269. The vertical axis is logarithmic; every marker is a saved post-step value.

C26 | Measured history: gamma = 3



Same n, initial vector, arithmetic and step count. Final norm: 22.3441606024. This vertical axis is linear. A changing parameter changes the matrix, not just the curve label.

AGH

Measured on the test platform

	`float`	`double`
Bytes	4	8
Significand bits	24	53
ϵ	$1.19\,10^{-7}$	$2.22\,10^{-16}$

What epsilon means

For these binary types, it is the gap from 1 to the next representable number. It is not an absolute error bound for an arbitrary calculation.

Measure, do not assume

Use `std::numeric_limits<T>` for radix, digits, epsilon and `is_iec559`. The C++ language does not require these exact type widths on every platform.

AGH

A qualified local model

For usual rounding and suitable finite normal-range operations,
$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta)$ with small $|\delta|$.
Overflow, underflow and relaxed optimization need separate care.

In this particular project

All A_{ij} and all iterate components are positive. The dot-product terms do not subtract, so **subtractive cancellation is not the explanation** for either norm curve.

What still matters

Entries are rounded; products and sums are rounded; tiny terms can be absorbed; errors propagate through later steps. Operation order can change the last digits.

$$D_{\mathrm{abs}} = \|x_a - x_{\mathrm{ref}}\|_2, \qquad D_{\mathrm{rel}} = \frac{D_{\mathrm{abs}}}{\|x_{\mathrm{ref}}\|_2} \quad (\|x_{\mathrm{ref}}\|_2 > 0)$$

A scale-aware acceptance rule

$D_{\mathrm{abs}} \leq \mathrm{atol} + \mathrm{rtol} \cdot \mathrm{scale}.$
First reject non-finite values. State the scale and tolerances. If the reference norm is zero, report the absolute discrepancy and mark the relative metric undefined.

Compare arrays, not only their norms

$(1,0)^T$ and $(0,1)^T$ both have norm 1 but differ by $\sqrt{2}$. Agreement with another implementation is evidence, not proof of exact accuracy.

AGH

The problem

Conditioning describes sensitivity of a specified mathematical problem to small changes in its inputs.

The algorithm

Numerical stability: does rounding cause excessive deviation from the intended mathematical result? Stable does not mean small output.

Read the lecture carefully

The lecture discusses stability through bounded output. Here we must distinguish bounded iterates from algorithmic error behaviour. More precision does not fix a wrong method or an ill-posed comparison.

$$x^{(k)} = A^k x^{(0)}, \qquad \rho(A) = \max_i |\lambda_i|$$

For this real symmetric positive-entry matrix

The all-ones start has a component along the positive dominant eigenvector. At the official size, $\gamma = 2$ has an amplifying dominant mode; $\gamma = 3$ is a contraction in the Euclidean norm.

Ask the right diagnostic question

Does the computed vector agree with an independently justified approximation to the *expected* result? A rising norm alone cannot answer that. R13–R15 give assumptions and bounds.

AGH

A — Precision

Compare full float arithmetic with double. Record storage, multiplication, accumulation and output policies. Mixed precision is a separate experiment.

B — Implementation agreement

Compare full manual and CBLAS outputs with exact tiny tests, finite checks, and absolute/relative discrepancies.

C — Compiler optimization

Use separate builds for `-O0`, `-O2`, `-O3` and optionally `-Ofast`. Strictly validate fast-math output in a separate process.

These are instructor-added extensions, not extra compulsory tasks. Complete the official project first.

$$\underbrace{A}_{M\times K}\underbrace{B}_{K\times N}=\underbrace{C}_{M\times N}, \qquad C_{ij}=\sum_{\ell=0}^{K-1} A_{i\ell}B_{\ell j}$$

A row meets a column

For each output position (i, j), take the dot product of row i of A and column j of B. The inner dimension K must agree.

The official square

$C = A^2$ means $A \times A$. It does **not** mean replacing every entry by A_{ij}^2. Use a separate output matrix.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Different inputs

$$AB = \begin{pmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{pmatrix}$$

$$= \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

The matrix square

$$A^2 = \begin{pmatrix} 1\cdot 1 + 2\cdot 3 & 1\cdot 2 + 2\cdot 4 \\ 3\cdot 1 + 4\cdot 3 & 3\cdot 2 + 4\cdot 4 \end{pmatrix}$$

$$= \begin{pmatrix} 7 & 10 \\ 15 & 22 \end{pmatrix}$$

```cpp
for (std::size_t i = 0; i < M; ++i) {
    for (std::size_t j = 0; j < N; ++j) {
        double sum = 0.0;
        for (std::size_t k = 0; k < K; ++k) {
            sum += a[i * lda + k] * b[k * ldb + j];
        }
        c[i * ldc + j] = sum;
    }
}
```

Complete arithmetic core; preconditions checked separately

Packed or padded row-major arrays; matching shapes; distinct output; finite inputs. For A^2, use the same input array for a and b, not for c.

This is the transparent i, j, k baseline. A cache-friendlier loop order remains an optional, separately labelled method.

Level	Typical work	Example
1	$O(n)$	Dot product: `ddot`
2	$O(n^2)$	Matrix–vector: `dgemv`
3	$O(n^3)$	Matrix–matrix: `dgemm`

Names and responsibilities

BLAS: Basic Linear Algebra Subprograms.
CBLAS: its C interface, callable from C++.
An implementation supplies the actual kernels; performance depends on that implementation and the environment.

Today only BLAS/CBLAS is needed for the library comparison. LAPACKE and FFTW belong to later tasks.

$$C \leftarrow \alpha\ \mathrm{op}(A)\,\mathrm{op}(B) + \beta C$$

```
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
            n, n, n,       // M, N, K
            1.0,           // alpha
            a.data(), lda, // first input and its stride
            a.data(), lda, // second input and its stride
            0.0,           // beta: overwrite old C
            c.data(), ldc);
```

For the official packed square

$M = N = K = \mathrm{lda} = \mathrm{ldc} = 2000$. The two read-only input pointers may coincide. The output pointer must not alias either input.

The header determines the integer interface type. Validate dimensions before converting a container size to that type.

AGH

Student branch

Run the environment probe before building the BLAS activity. The activity itself remains incomplete until you implement its functions.

```
make check-env
make activity4-blas
./bin/activity4_blas
```

Equivalent explicit build from the starter root

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -O2 \
    -DSTUDENT_WITH_BLAS -Iinclude src/activity4.cpp \
    -o bin/activity4_blas -lblas -lm
```

The probe expects $\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$ for AB; the activity checks $\begin{pmatrix} 7 & 10 \\ 15 & 22 \end{pmatrix}$ for A^2.

Report missing headers/linkage; do not install on managed machines.

www.agh.edu.pl

AGH

A4a–b: establish correctness

Implement `square_manual` and `square_blas`. Check the 2×2 fixture, with nonzero prefilled output. Verify the whole result, not just one entry.

A4c–d: extend and measure

Build the official gamma matrix and compute A^2. Add full-array discrepancy checks and `steady_clock` timing. Start at small/moderate sizes; run $n = 2000$ only after estimating cost and confirming resource policy.

```
make activity4
./bin/activity4
make activity4-blas
./bin/activity4_blas
```

Record n, γ, flags, library, repeats and thread policy. A tiny PASS message does not certify the unimplemented large benchmark.

C40 | Validate the entire matrix

$$D_F = \|C_{\text{manual}} - C_{\text{BLAS}}\|_F, \qquad \|C\|_F = \sqrt{\sum_{ij} |C_{ij}|^2}$$

Acceptance, not blind equality

First reject non-finite entries. Use a documented absolute-plus-relative criterion on the full logical matrix, excluding padding. Also inspect the maximum component discrepancy.

Test cases that reveal hidden mistakes

Use nonsymmetric rectangular matrices and padded strides. The official symmetric matrix alone can hide a transpose/layout error. Equal norms do not prove equal matrices.

Small exact fixtures provide an independent anchor. BLAS-versus-manual agreement is a discrepancy check, not exact arithmetic.

```cpp
const auto start = std::chrono::steady_clock::now();
// Execute the selected multiplication kernel here.
const auto stop = std::chrono::steady_clock::now();
const double seconds =
    std::chrono::duration<double>(stop - start).count();
```

Inside the interval

Execute only the selected multiplication kernel. Include any reset that is intrinsic to this method.

Outside the interval

Prepare and allocate first. Validate the result, evaluate its checksum and write all output afterward.

A monotonic clock avoids wall-clock adjustments. It does not guarantee enough resolution for an extremely tiny operation.

AGH

Hold the problem fixed

Same matrix size, gamma, stored input values, precision and operation. Record the actual BLAS backend and requested/observed thread policy.

Repeat and qualify

Warm up, then record repeated measurements. Report the median and spread. Moderate runs here use five repeats; the expensive $n = 2000$ cases use two, with that limitation stated.

Do not generalize the local result

Cache behaviour, compiler decisions and concurrent load affect timings. Application flags do not rebuild a precompiled BLAS library.

γ	Method	Median / s	Range / s
2	Manual i, j, k	13.163	12.416–13.909
2	CBLAS	4.763	4.471–5.054
3	Manual i, j, k	13.388	12.617–14.159
3	CBLAS	5.051	4.988–5.114

Conditions of this measurement

$n = 2000$, binary64; GCC 14.2, `-O2`; Debian reference BLAS 3.12.1; one observed process thread. One warm-up and two timed samples per method/case. Kernel-only wall time.

Local authoring container, **not Taurus/classroom**. Median manual/BLAS ratios: about 2.76 for $\gamma = 2$ and 2.65 for $\gamma = 3$. The sample range is not a statistical confidence interval.

C44 | Submit a reproducible project

AGH

Activity ZIP

Your source files.
Required numerical data.
Recommended README:
compile and run commands.
Keep each project identifiable.
Test the ZIP after creating it.

Separate report PDF

Method: about one page.
Task: about half a page.
Results: labelled plots/tables with short comments.
Conclusions: about half a page.

Published timing rule

Submit the report within 7 days after the relevant laboratory. The course page states a 20-point deduction for each subsequent week of delay. Confirm the actual UPeL group and assignment deadline.

C45 | Exit check: explain your choices

Q1–Q2: representation and algorithm

What does `lda` measure? Why must a row sum be reset? Explain why the official iteration uses two vectors.

Q3–Q4: reliability

Does a growing norm prove instability? Does manual/BLAS agreement prove exact accuracy? State one independent check and one limitation.

Next connection

These kernels become building blocks for linear systems and eigenvalue methods. Today's project remains the specified vector histories and matrix-square comparison.

AGH

Use this appendix selectively

R02–R05: layouts, common bugs and robust norms.
R06–R12: independent checks, precision and error interpretation.
R13–R16: spectral explanation and the power-method connection.
R17–R19: compiler experiments, debugging and sources.

Scope remains unchanged

These are study aids and optional extensions, not twenty more compulsory live-lecture slides. The official task remains the two norm histories and the manual/CBLAS matrix-square comparison.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad (2 \text{ rows, } 3 \text{ columns})$$

Packed row-major

$\mathrm{lda} = 3$.
Storage: $[1, 2, 3, 4, 5, 6]$.
$A_{1,2} = 6$ has offset
$1 \cdot 3 + 2 = 5$.

Packed column-major

$\mathrm{lda} = 2$.
Storage: $[1, 4, 2, 5, 3, 6]$.
$A_{1,2} = 6$ has offset
$1 + 2 \cdot 2 = 5$.

Use more than one entry

The last entry happens to have offset 5 in both packed layouts. But $A_{0,1} = 2$ has offsets 1 and 2. One coincident offset cannot verify a layout.

$$A_{2\times3}B_{3\times2} = C_{2\times2}, \qquad M = 2,\ N = 2,\ K = 3$$

Row-major array	Logical shape	Chosen stride	Allocated slots
A	2×3	$\mathrm{lda} = 5$	10
B	3×2	$\mathrm{ldb} = 4$	12
C	2×2	$\mathrm{ldc} = 3$	6

A complete result check

$A = [1, 2, 3; 4, 5, 6]$, $B = [7, 8; 9, 10; 11, 12]$ give $C = [58, 64; 139, 154]$. In the padded output, $C_{1,1}$ occupies offset $1 \cdot 3 + 1 = 4$. Padding remains untouched.

Hazard	Symptom and correction
Unsigned `i-j`	Wraparound when $i < j$; convert each index before subtracting.
32-bit `d*d*d*d`	Overflow at large separations; use safe floating or sufficiently wide arithmetic.
`d^4`	XOR, not exponentiation; square twice for this integer fourth power.
Integer division	`1/2` is zero in integer arithmetic; use `1.0` and the intended floating type.

A targeted test

Check distances 0, 1, 2 and 1999 for both gamma values. Small examples alone do not expose the fourth-power overflow.

AGH

Mathematical scaling idea

Let $s = \max_i |x_i|$. If $s = 0$, the norm is zero. Otherwise,

$$\|x\|_2 = s\sqrt{\sum_i (x_i/s)^2}.$$

The ratios are at most one in magnitude.

Why it matters

For $x = (10^{200})$ in binary64, the true norm 10^{200} is representable. Forming x_0^2 first overflows. A scaled sum avoids that intermediate square.

The reference uses a one-pass scale/sum-of-squares update. It still fails explicitly when the final norm itself is not representable.

AGH

Exact small anchors

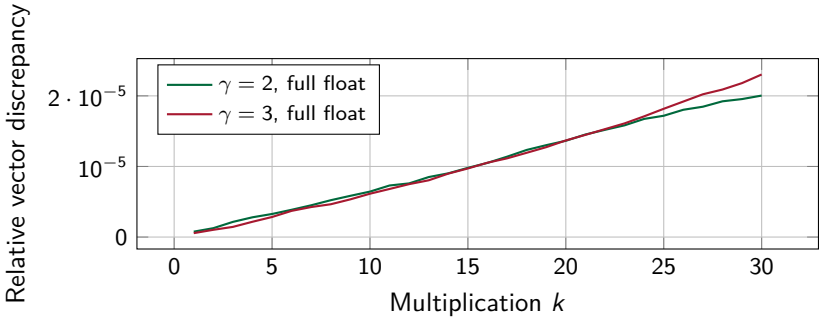
Integer-valued dot, matvec, AB, A^2 and rectangular fixtures. For $n = 1$, $x^{(k)} = \gamma^{-k}$. For $n = 2$, use the analytic all-ones eigenvector.

Checks actually performed

At $n = 8, 16$: independent Decimal recurrences at 80 and 120 digits, for both exact rational and stored-binary64 matrices.
At $n = 2000$: all 30 full vectors compared with a diagonal-offset traversal.

Know the reference target

The exact mathematical matrix and its rounded stored version are different problems. A second library call is not automatically an independent exact oracle.



Both curves compare **full float storage/products/accumulation** with double at $n = 2000$. Inputs begin from the same binary64 matrix, converted once for float. Mixed double accumulation is a separately recorded policy. No curve is forced to be monotone.

```
std::cout << std::setprecision(17) << 0.1;
// On the tested binary64 platform: 0.10000000000000001
```

Representation versus serialization

1/10 has no finite binary expansion. The stored binary64 value is close to, but not exactly, 1/10. Printing enough digits allows that stored value to be reconstructed.

Actual type information

`max_digits10`: 9 for float, 17 for double here.
`long double`: 64 significand bits and 16 storage bytes here; that does not make it an exact oracle.

AGH

Two orders; the same exact sum

$a = [10^{16}, 1, -10^{16}]$, $b = [10^{16}, -10^{16}, 1]$.
Both exact sums equal 1. This is a **separate signed example**, not the positive gamma dot product.

Tested method	Order a	Order b	Exact
Sequential	0	1	1
Pairwise	0	0	1
Neumaier compensation	1	1	1

www.agh.edu.pl

```
double sum = 0.0, correction = 0.0;
for (double value : x) {
    const double next = sum + value;
    if (std::abs(sum) >= std::abs(value))
        correction += (sum - next) + value;
    else
        correction += (value - next) + sum;
    sum = next;
}
return sum + correction;
```

Neumaier variant; optional study

The correction tracks low-order information lost from the main sum. It improves many sums, not all computations. Compile this strict experiment without reassociation/fast-math transformations.

$$x^{(k+1)} = Ax^{(k)}, \qquad \widehat{x}^{(k+1)} = \widehat{A}\widehat{x}^{(k)} + \eta_k$$

Define the accumulated error

Let $e_k = \widehat{x}^{(k)} - x^{(k)}$. Add and subtract $A\widehat{x}^{(k)}$ to obtain

$$\boxed{e_{k+1} = Ae_k + (\widehat{A} - A)\widehat{x}^{(k)} + \eta_k}.$$

Three sources

Ae_k: previous error propagated.
$(\widehat{A} - A)\widehat{x}^{(k)}$: rounded matrix entries.
η_k: local multiply/add rounding.

AGH

A fully specified example

Solve $2x = 1$. Exact solution: $x = 0.5$. Suppose a computed answer is $\widehat{x} = 0.49$.

Forward error

Change in the answer:
$|0.49 - 0.5| = 0.01$.
Relative to $|x| = 0.5$,
the forward error is 0.02.

Backward error in the RHS only

Keep coefficient 2 fixed.
$2(0.49) = 0.98$.
The computed answer exactly solves the nearby problem with $b = 0.98$, a relative RHS change of 0.02.

The residual is $1 - 2(0.49) = 0.02$. State which input may change; a different perturbation model defines a different backward-error problem.

$$Aq_r = \lambda_r q_r, \quad q_r^T q_s = \delta_{rs}, \qquad x^{(0)} = \sum_r c_r q_r$$

Because this matrix is real and symmetric

$$A^k x^{(0)} = \sum_r c_r \lambda_r^k q_r, \qquad \|A^k x^{(0)}\|_2^2 = \sum_r c_r^2 |\lambda_r|^{2k}.$$

An excited mode with $|\lambda_r| > 1$ can eventually dominate.

Do not overgeneralize

Not every start excites every mode. Entrywise positivity makes the all-ones start overlap the positive dominant mode here. Arbitrary nonsymmetric matrices need more careful transient analysis.

$$q = \frac{1}{\sqrt{n}}(1, \ldots, 1)^T, \qquad \lambda_{\max} \geq q^T A q$$

Keep only the diagonal and first off-diagonals

All omitted entries are positive. Therefore

$$q^T A q \geq \frac{1}{2} + \frac{2(n-1)}{3n}.$$

For $n = 2000$, the bound is $1.166333\ldots > 1$.

A useful small-size contrast

For $n = 2$, the all-ones eigenvalue is $1/2 + 1/3 = 5/6 < 1$. That small test decays. The gamma label alone does not establish the trend for every dimension.

$$\rho(A) \le \|A\|_\infty \le \frac{1}{3} + \frac{1}{2} + \frac{2}{19} + 2\sum_{d=3}^{\infty} \frac{1}{d^4}$$

Bound the remaining tail

$$\sum_{d=3}^{\infty} d^{-4} \le \frac{1}{81} + \int_3^{\infty} x^{-4}\, dx = \frac{2}{81}.$$

$$\rho(A) \le \frac{1}{3} + \frac{1}{2} + \frac{2}{19} + \frac{4}{81} \approx 0.98798 < 1.$$

Since A is real symmetric, $\|A\|_2 = \rho(A)$. Thus $\|Ax\|_2 \le 0.98798\ldots \|x\|_2$ for every vector. Finite boundary rows only reduce the row-sum bound.

AGH

This official project

$x^{(k+1)} = Ax^{(k)}$.
Preserve the changing norm.
Study amplification or decay.
No eigenvalue solver is required.

A later normalized iteration

$v^{(k+1)} = Av^{(k)}/\|Av^{(k)}\|_2$.
Preserve a direction while controlling scale.
Useful under appropriate spectral and starting-vector conditions.

Do not substitute one experiment for the other

Normalization sets the vector length to one and removes the official observable.
Directional convergence and norm growth are different questions.

Application flag	Median / ms	Strict check
-O0	10.206	PASS
-O1	1.881	PASS
-O2	2.039	PASS
-O3	1.982	PASS
-Ofast	2.468	PASS

A fixed, small comparison

GCC 14.2; manual i, j, k kernel; $n = 128$, $\gamma = 2$; one warm-up and five timed repeats. Each build has a separate executable and output directory.

`-Ofast` permits relaxed floating-point transformations. Its dumped matrix was checked in a separate strict Python process. These local values are not a universal speed ranking; BLAS was not rebuilt.

R18 | Debug the first failing stage

Symptom	First useful action
Header not found	Identify machine; check compiler include path and installed interface.
Linker error	Check actual library and flags after source/object inputs.
Wrong tiny answer	Check shape, stride, accumulation and aliasing.
Non-finite output	Inspect entry types, denominator, range and first failing step.
Wrong row count	Trace post-step indexing and separate initial-state output.
Plot fails	Check CSV header, numeric columns, finite values and input path.


THANK
YOU!